Curated File Guide: index.html

The public portfolio shell: metadata, semantic page structure, runtime hooks, visible sections, and script/style entry points.

Before The Code: File Overview

Abstract

index.html is the public website shell. It is the first file a recruiter or visitor receives from the static host. It defines the document metadata, search-engine hints, social-card metadata, favicon and app icons, structured data, visible page sections, semantic landmarks, and the fixed DOM ids that script.js later fills with real portfolio content. The file is intentionally light on final personal content because the builder writes most custom data into projects.json and script.js renders it dynamically.

The most important idea is that index.html is an interface contract. It tells the browser, search engines, social media previews, accessibility tools, styles.css, and script.js where things are and what each major region means. If script.js is the behavior layer, index.html is the skeleton that gives that behavior safe attachment points.

Introduction

The document begins with standard HTML setup: language, character set, viewport, description, robots instructions, author, and the AI endpoint metadata. These are not decorative tags. They affect how the page is indexed, how it scales on phones, how icons appear, and where the AI chat sends requests. After the metadata, index.html defines the visible page from top to bottom: header, hero, fun facts, builder download, summary metrics, projects, resume, dynamic sections, process, contact/AI, and footer.

Most of the actual portfolio content is not typed directly in this file. That is deliberate. The builder is supposed to let users change names, images, resumes, project categories, links, and rich text without editing HTML manually. Therefore index.html supplies stable placeholders such as #hero-title, #project-grid, #resume, #dynamic-sections, #contact, and #ask-ai. script.js finds those placeholders and fills them from the catalog.

Background Information

A static portfolio can be hosted cheaply and reliably if the page can be built from HTML, CSS, JavaScript, and JSON. index.html is designed for that model. It avoids server-only assumptions and includes assets with relative paths. It loads electronics-search.js before script.js because script.js can use the electronics search vocabulary when building project search terms.

The file also contains schema.org JSON-LD. That structured data gives search engines a machine-readable hint that the page is a ProfilePage about a Person. The builder later updates the visible content, but this baseline makes the page understandable even before dynamic content fully loads.

The page uses semantic sections and labels because public portfolio viewers include recruiters, mobile users, browser accessibility tools, and search crawlers. A good static shell gives all of them landmarks. For example, the projects section is labeled by projects-title, the resume section is hidden until a resume URL exists, and the AI assistant has a labelled panel so it can be reached from navigation and screen readers.

The file is intentionally not a finished personal biography. It is a reusable public shell. A fresh install should not hardcode Maurice-specific content, while Maurice's own built portfolio can still publish his profile through the catalog. That is why placeholder labels such as Portfolio Owner, Portfolio, and generic engineering copy exist here. The builder-generated catalog is the layer that specializes the site.

The HTML also controls load order. CSS loads before the page renders so layout and colors are available immediately. electronics-search.js loads before script.js because script.js may call the electronics keyword helper while building the search index. script.js loads at the end of the body so the DOM elements already exist by the time it queries them. If script.js loaded too early without deferring, many document.querySelector calls would return null.

Purpose Of The File

The purpose of index.html is to guarantee that the public portfolio has a stable starting structure. Without it, script.js would have nowhere reliable to render projects, no header to navigate from, no AI panel to attach events to, no resume viewer to populate, no contact band to update, and no metadata to help browsers or search engines understand the page.

It also separates permanent page structure from changeable portfolio content. The structure changes slowly; the content changes whenever the builder saves and publishes. That separation is what allows the builder to generate portfolio updates without rewriting the entire website architecture each time.

The file is also the public accessibility foundation. Section headings, labels, aria attributes, hidden states, object fallback text, and semantic main/header/footer landmarks make the site more understandable to assistive technology. script.js can only enhance that foundation; it cannot rescue a shell that has no stable names or landmarks.

Another purpose is failure tolerance. If projects.json fails to load, the page should still show a header, hero, search area, process section, and footer. If the resume is not configured, the resume section remains hidden. If the profile image is missing, the image elements stay hidden. This is what keeps a partially configured portfolio from looking broken.

Primary Responsibilities

The first responsibility is document identity. The head contains title, description, canonical URL, social tags, icons, AI endpoint, and structured data. These values determine how the page appears to crawlers, shared links, browser tabs, and mobile install surfaces.

The second responsibility is visible layout landmarks. The body defines the header, main content regions, sections, and footer. styles.css controls how they look, and script.js controls most dynamic filling.

The third responsibility is runtime hooks. Every id and class that script.js queries is part of a contract. Removing #project-grid, #project-search, #ask-ai, #contact, #resume, or #dynamic-sections would break the corresponding rendering logic.

The fourth responsibility is graceful fallback. The page contains placeholder text and hidden sections so the website has a sensible shape even before projects.json finishes loading. If a profile image or resume does not exist, those areas remain hidden instead of showing broken controls.

The fifth responsibility is hosting compatibility. The document uses ordinary relative paths and static assets. There is no server-side template language, no database call, and no build step required at request time. That is why the same page can be served from GitHub Pages, Cloudflare, or a local preview server.

The sixth responsibility is public trust. The page exposes recruiter-facing areas only: projects, resume, process, contact, download builder, and AI assistant. It does not include local builder controls, Git publishing buttons, compiler controls, or editing widgets. That separation protects the private builder workflow from public visitors.

Important Elements And Why They Matter

The header is the main navigation surface. Its brand link returns to the top, the nav links jump to AI, projects, resume, process, and contact, and the avatar link can take the visitor to contact information. The brand image and text are later customized by script.js.

The hero section is the first impression. It contains a hidden hero image, overlay shade, eyebrow, title, copy, project/resume buttons, GitHub button, and Ask AI jump link. These are all visible near the top because recruiter-facing content should not require deep scrolling.

The project section is the core portfolio directory. It includes a search input, filter controls, and the empty project grid that script.js fills. The search input is deliberately placed in the section heading because search is a primary navigation method, not an afterthought.

The contact band is last. It holds contact information and the AI assistant panel. The AI lives in this area because it is a helper for exploring the portfolio, while the contact section remains the final page region.

The hidden attributes are part of the contract. They do not mean the feature is dead; they mean script.js will reveal the feature only when the catalog contains the required data. This prevents empty resume viewers, empty avatars, and profile images without sources from appearing to visitors.

The IDs are also part of the contract. #project-grid, #project-search, #project-filters, #resume, #dynamic-sections, #contact, #ask-ai, #ai-assistant-form, #ai-assistant-input, and #ai-assistant-log are the anchors script.js expects. Renaming them would not only change HTML; it would break runtime behavior.

Code Walkthrough

The walkthrough below groups related functions where they form one idea. Small helpers are explained together when that is clearer than pretending each one is a separate chapter. Larger functions receive direct explanations of their parameters, internal objects, side effects, return values, and why they are necessary.

Head metadata and search identity

The head makes the page understandable before JavaScript renders anything.

charset, viewport, description, robots, author, and canonical link

The charset tells the browser to interpret the file as UTF-8. The viewport makes the page scale correctly on phones. The description gives search engines a concise page summary. robots tells crawlers that indexing is allowed. author is a generic placeholder updated by published content or metadata later. The canonical link is empty in the template because the final domain can vary.

portfolio-ai-endpoint meta tag

This meta tag gives script.js a backend URL for the portfolio AI assistant. assistantEndpoint reads it before falling back to /api/portfolio-ai. Keeping the endpoint in metadata lets the same JavaScript bundle run locally and publicly while pointing to different AI backends.

Open Graph and Twitter card tags

These tags control how the portfolio looks when shared on social platforms or messaging apps. The defaults are generic because the builder can later fill real owner, title, image, and description values.

favicon and touch-icon links

These links tell browsers and mobile operating systems which icons to show in tabs, bookmarks, install prompts, and home-screen shortcuts. They point at the OMB asset set so the portfolio has a recognizable identity.

ProfilePage JSON-LD script

This structured data gives search engines a machine-readable summary of the page as a profile page about a person. The values are conservative placeholders. The important part is the shape: Person, name, image, URL, sameAs, and knowsAbout. It gives crawlers context even though the visible profile is populated dynamically.

Header and navigation contract

The header gives visitors immediate navigation and gives script.js stable places to insert brand/profile visuals.

header#top and brand link

The header has id top so Back to top links can target it. The brand anchor points to #top and contains the brand image, OMB engraving, title, and subtitle. script.js later replaces brand text and image from profile data. If these classes disappeared, renderProfile would lose its brand hooks.

site-nav links

The navigation links jump to Ask AI, Projects, Resume, Process, and Contact. These anchors match section ids later in the file. They are simple hash links so the page works on static hosting and does not need a router.

header-avatar

The avatar link starts hidden because not every portfolio has a profile image. renderProfile reveals it and sets the image when profile data provides one. Its href points to contact so the image acts as a fast route to contact information.

Hero, fun facts, download, and metrics

These are top-of-page sections that help a visitor orient quickly.

hero section

The hero contains a hidden image, shade overlay, eyebrow, h1 title, copy, action buttons, optional GitHub button, and Ask AI jump link. script.js updates the text and images. The section is labelled by hero-title, which improves accessibility and gives the document a clear main heading.

fun-facts-callout

This hidden section becomes visible only when the catalog includes fun facts or rich fun fact blocks. It sits near the top so personality appears before deep project scrolling.

builder-download-section

This section exposes the latest Windows installer link. It is public because visitors may want to clone/install the builder app. The href points to the stable latest release asset. The download attribute suggests download behavior, but the browser and GitHub ultimately decide how the file is handled.

summary-band metrics

The metrics display project count, category count, and artifact promise. script.js updates the first two after loading projects.json. The third is static because the builder portfolio consistently supports source, diagrams, and reports.

Projects, resume, dynamic sections, process, contact, and AI

These sections are the public portfolio body. Most content is populated dynamically.

projects section

This is the main project directory. It has a heading, search input, filter bar, and empty project-grid. script.js builds filter buttons, project categories, project cards, parsed section buttons, search results, and nested section windows from this area.

resume section

The resume section starts hidden because a fresh portfolio may not have a resume. renderProfile reveals it, sets the PDF object data, and updates Open/Download links when profile.resumeUrl exists. The object element gives an in-page PDF viewer when the browser supports it.

dynamic-sections container

This div is where user-created main portfolio sections render. The builder can add personal life, professional profile, social links, hobbies, or other lighter sections without editing index.html.

process section

The process section is a stable explanatory band describing problem/constraints, build artifacts, and results/reflection. It gives the page a professional engineering narrative even before specific custom sections are added.

contact-band and ask-ai panel

The contact band is intentionally last. It holds profile photo, contact details, external links, and the AI assistant. The AI form, log, input, status, and clear button have fixed ids because script.js attaches the chatbot behavior directly to them.

footer and script loading

The footer provides Back to top and copyright year. The scripts load electronics-search.js before script.js so the search vocabulary is available when script.js builds the index. Version query strings help browsers fetch updated CSS and JS after publishing.